

Capture ***this** with initializer

Contents

1. [Revision history](#)
2. [Summary](#)
3. [Discussion](#)
4. [Proposed wording](#)
5. [Non-proposal](#)

Revision history

- P0637R0: Initial proposal.

Summary

We propose to allow the lambda by-value capture of ***this** to rebind **this** to arbitrary objects: `[*this = std::move(x)](){}`

Ordinary capture	*this capture
<pre> struct X { void f(int n); void g() { // Capture copy of current object X that = *this; [that] (int n) { that.f(n); } // C++11 [self = *this] (int n) { self.f(n); } // C++14 } void h() { // Capture copy of different object X that = X(); [that] (int n) { that.f(n); } // C++11 [self = X()] (int n) { self.f(n); } // C++14 [self = std::move(*this)](int n) { self.f(n); } // C++14 } }; </pre>	<pre> struct X { void f(int n); void g() { // Capture copy of current object [*this] (int n) { f(n); } // C++17 } void h() { // Capture copy of different object [*this = X()] (int n) { f(n); } // New! [*this = std::move(*this)](int n) { f(n); } // New! } }; </pre>

In particular, this feature “adds move semantics to C++17’s value-capture of ***this**”. Situations that motivated value-capture (see [P0018R3](#)), such as asynchronous dispatch, arise when the current class instance does not outlive the desired lambda, in which case the lambda needs a copy of the current instance. However, in typical cases where only one such “asynchronous hand-off” happens, a copy is not actually required, and it suffices to *move* the instance data along into the lambda.

Discussion

C++11 introduced lambda expressions that could capture named entities from their environment by value. C++14 extended this facility to allow capture by value of arbitrary expressions. C++17 introduced the ability for lambda expressions that appear in the definition of a non-static class member function to rebound the meaning of the `this` keyword inside the lambda body: The rebound `this` would refer to a private copy of the class instance that is captured by the lambda expression and stored within the closure object (P0018R3):

```
struct X {
    void g(int n);
    void f() {
        auto h = [self = *this](int n) { self.g(n); };
        // Same as:
        auto h = [*this](int n) { this->g(n); }; // "this == &__self" for a secret value __self
    }
};
```

The extension provided by P0018R3 looks simple, but is conceptually profound: For the first time, the meaning of `this` can vary inside a single class's scope and be rebound by the user. Once we are comfortable with this situation, it is only natural to ask for ability to rebound `this` to *any* captured value of the right type. Just as C++14 added value capture from expressions to C++11 value capture, we propose to add `*this` capture from expressions to C++17 `*this` capture.

Unlike the *simple-capture* "`*this`", which would be equivalent under this proposal to "`*this = *this`", the *init-capture* form is not constrained to appear in a non-static class member, since it does not require a prototypical instance to copy from, and we can allow it in static member functions just as well.

Example:

```
struct X {
    std::unique_ptr<int> p_;
    void g(int n) const;

    auto f() && {
        return [*this = std::move(*this)](int n) { g(n + *p_); }; // move the current object
    }

    void f(X x) {
        return [*this = std::move(x), m = *p_]() { g(m + *p_); }; // move some other object
    }

    static auto CallMyX(int m) {
        return [*this = X{std::make_unique<int>(m)}](int n) { g(n + *p_); } // use a newly
        created instance
    }
};
```

Proposed wording

Change 8.1.2 [expr.prim.this] paragraph 2 as follows.

[...] It shall not appear before the optional *cv-qualifier-seq* and it shall not appear within the declaration of a static member function (although its type and value category are defined within a static member function as they are within a non-static member function), except in the body of a lambda expression that captures `*this` [Note: necessarily via init-capture – end note]. [Note: This is because [...]]

Change the grammar in 8.1.5.2 [expr.prim.lambda.capture] as follows.

init-capture:
 identifier initializer
 & *identifier initializer*
 * this initializer

Change paragraph 6 as follows.

An *init-capture* that is not * this initializer behaves as if it declares and explicitly captures a variable of the form “`auto init-capture ;`”. An *init-capture* * this initializer behaves as if it declares and explicitly captures a variable of the form “`auto __this initializer ;`”. The ~~whose~~ declarative region of this variable is the lambda-expression’s compound-statement, except that:

- if the capture is by copy (see below), the non-static data member declared for the capture and the variable are treated as two different ways of referring to the same object, which has the lifetime of the non-static data member, and no additional copy and destruction is performed, and
- if the capture is by reference, the variable’s lifetime ends when the closure object’s lifetime ends.

[*Note*: This enables an *init-capture* like “`x = std::move(x)`”; the second “`x`” must bind to a declaration in the surrounding context. – *end note*]

Change paragraph 8 as follows.

[...] If `*this` is captured by a local lambda expression, its nearest enclosing function shall be a non-static member function. If `__this` is captured, the nearest enclosing function shall be a member function and the type of `__this` shall be the same as that of the enclosing class. If a lambda-expression [...]

Change paragraph 11 as follows.

[...] If `*this` is captured by copy or `__this` is captured, each odr-use of `this` is transformed, respectively, into a pointer to the corresponding unnamed data member or to the `__this` member of the closure type, cast (8.4) to the type of `this`. [...]

Non-proposal

This proposal is limited to value capture of `*this`. A natural question is why we would not also allow *reference* capture of `*this` to rebind `this`. The reason is that such an extension seems much more difficult, and it seems less useful to the author.

Let us briefly examine the problems. A reference *init-capture* currently starts with `&`. The natural syntax for reference *init-capture* of `*this` would be “`[&*this = x]`”. This syntax may look surprising at first sight, but it is natural if we think of `*this` as the fundamental object. But then we would end up with two separate syntaxes, `[this]` for the *simple-capture* and `[&*this = x]` for the *init-capture*, and `[this]` would be equivalent to `[&*this = *this]`. If we were tempted to replace `&*this` with just `this` for backwards “consistency”, we would end up with paradoxical syntax like `[this = *this]`. Alternatively, or additionally, we could consider a separate pointer capture `[this = &x]` that reseats `this`. We believe that such extensions should be discussed in depth in a separate proposal.